A Benchmark of Five Machine-Learning Architectures for Daily Trading-Signal Classification on US Tech Equities

Author Name[1][0000-0000-0000-0000]

Institution, City, Country
author@institution.edu

Abstract. We benchmark five supervised classifiers — multinomial logistic regression (LR), XGBoost, a bidirectional Long Short-Term Memory network (LSTM), a 1-D Convolutional Neural Network (CNN), and a hybrid CNN-LSTM — on the task of predicting daily BUY/HOLD/SELL signals for seven large-cap US technology stocks (AAPL, NVDA, TSLA, AMZN, MSFT, GOOGL, META). The label is derived from the rolling 30/70-percentile of the one-day forward log-return over a 252-day window, producing an adaptive ground truth that is robust to bull/bear regimes. We use 61 indicators built from daily open-high-low-close-volume (OHLCV) data plus SPY and VIX context, all obtained for free from Yahoo Finance. Three temporal splits share the same test year (2025) but differ in the number of training years (10, 6 and 4). Each model is trained both *globally* (one model for all tickers) and *per ticker*, yielding 120 controlled runs. We report F1-macro alongside three backtest metrics — win rate, profit factor and maximum drawdown — to capture both classification and economic quality. LR with elasticnet regularization achieves the highest average F1-macro (0.399) and the best Sharpe ratio in the principal experiment (+0.76), outperforming the deep architectures. A separate ablation shows that statistical feature filtering (Pearson, VIF, Spearman, SHAP) hurts performance in 11 of 15 cases: internal model regularization is more effective than external pre-selection. We also find that ticker volatility is the dominant factor behind cross-asset performance variation, irrespective of the model class.

Keywords: Trading signals · Classification · Deep learning · Logistic regression · XGBoost · LSTM · CNN.

1 Introduction

Short-term forecasting of equity prices is one of the most studied problems in quantitative finance. The Efficient Market Hypothesis [1] predicts that meaningful predictability should not exist; in practice, recent surveys [2,4] show that machine-learning models can extract a small but exploitable signal from technical indicators. The literature, however, suffers from three recurring limitations:

(i) comparisons usually involve only one or two models; (ii) the reported metric is often accuracy or RMSE, which do not translate cleanly to trading performance; and (iii) feature pre-selection by univariate statistical tests is applied without checking whether it actually helps. This paper addresses these three points jointly.

We compare five representative models — a linear baseline (LR), a gradient-boosted tree model (XGBoost), and three deep-learning architectures (LSTM, CNN, CNN-LSTM) — under an identical protocol across 120 controlled runs (5 models $\times$ 3 splits $\times$ 8 modalities). All runs share the same test year (2025) for a fair comparison, and only the amount of training history differs (10, 6 or 4 years). For every run we report F1-macro together with three backtest metrics: win rate, profit factor and maximum drawdown.

Contributions.

1. A reproducible pipeline using only free Yahoo Finance data.
2. A five-model benchmark with 120 runs and dual evaluation (classification + economics).
3. Temporally aligned splits (same test year) that isolate the effect of training-window size.
4. An empirical assessment of statistical feature selection (Pearson, VIF, Spearman, SHAP) against using all features.
5. A discussion of why ticker volatility, rather than model choice, explains most of the cross-asset variance in Sharpe.

2 Related Work

Sezer et al. [2] review more than 140 papers on deep learning for financial time series and observe that LSTM and CNN dominate the published architectures, while linear baselines are rarely included for comparison. Fischer and Krauss [3] apply LSTM to S&P 500 constituents and report a positive abnormal return, although later replications have raised concerns about look-ahead bias in the percentile-based label definition. Patel et al. [5] compare SVM, ANN and random forest on the Indian market and find that well-regularized linear and tree models match deep networks at daily horizons. For tree-based methods, XGBoost [6] remains a strong baseline; we tune it with the Tree-structured Parzen Estimator (TPE) sampler of Optuna [7]. The literature does not appear to contain a side-by-side comparison of LR, XGBoost and three different deep architectures under one experimental protocol, with both classification and economic evaluation; that is the gap addressed here.

3 Data and Target

3.1 Assets and source

We use the seven largest US technology stocks by market capitalization at the end of 2024: AAPL, NVDA, TSLA, AMZN, MSFT, GOOGL and META. Daily

Table 1: Feature categories. All indicators are derived from OHLCV; SPY/VIX provide market context.

Category	Count	Examples
Log returns	5	ret_1d, ret_2d, ret_3d, ret_5d, ret_10d
Momentum	4	mom_5d, mom_20d, mom_60d
Moving averages	9	dist_ma$\{10, 20, 50, 200\}$, 3 crossovers, slope MA20
Volatility	8	ATR(14), realized vol 5/10/20/60d, 2 ratios
Oscillators	9	RSI(7,14), MACD/signal/hist, Stoch K/D/diff, Williams %R
Volume/flow	8	OBV, VWAP distance, CMF(20), MFI(14), vol_ratio
Candle patterns	7	body/range, upper/lower shadow, gap, HL ratio
Seasonality	5	weekday and month (sine/cosine), week-of-month
Market context	6	SPY return/vol/momentum, VIX level/change/normalized
Total	**61**	

OHLCV data from 2013-12-31 to 2025-12-29 are downloaded with the `yfinance` library, with adjustments for splits and dividends. Market context is added through SPY (S&P 500 ETF) and ^VIX (the CBOE Volatility Index). No paid data source is used; the entire pipeline runs on freely available Yahoo Finance data.

3.2 Target construction

Let C_t be the close price on day t. The forward log-return is

$$r_{\text{fwd}}(t) = \ln \frac{C_{t+1}}{C_t}. \tag{1}$$

Over a rolling window of $W = 252$ trading days, we compute the 30th and 70th percentiles of r_{fwd} using only data available before day t. The label is

$$y_t = \begin{cases} \text{BUY (2)} & \text{if } r_{\text{fwd}}(t) \geq q_{70}(t-1), \\ \text{SELL (0)} & \text{if } r_{\text{fwd}}(t) \leq q_{30}(t-1), \\ \text{HOLD (1)} & \text{otherwise.} \end{cases} \tag{2}$$

By construction the class distribution is close to 30/40/30 in every market regime, removing the need to set an arbitrary $\alpha\sigma$ threshold. The shift by one day in the quantile computation prevents look-ahead.

3.3 Features (61 in total)

We compute the indicators in Table 1. All values are derived from daily OHLCV plus SPY and VIX. Seasonality variables are encoded as sine/cosine pairs to preserve periodicity. No fundamentals, news or option data are used.

Table 2: Temporal splits. All three share the same test year (2025) for direct comparison.

Experiment	Train	Validation	Test	Train years
A	2014–2023	2024	2025	10
B	2018–2023	2024	2025	6
C	2020–2023	2024	2025	4

3.4 Temporal splits

We define three splits that share the same out-of-sample year (2025) and only differ in the amount of training history (Table 2). This design isolates the effect of training-window length, at the cost of a single market regime (Section 9).

4 Models

Logistic regression (LR). A multinomial logistic regressor with elasticnet penalty [11] ($\ell_1 + \ell_2$, SAGA solver). The grid over $C \in \{0.01, 0.1, 1\}$ and l_1-ratio $= 0.5$ is searched on validation; class weights are balanced; multi-seed averaging combines five seeds by majority vote.

XGBoost. Gradient-boosted trees with `multi:softprob` objective and `hist` method on GPU. Hyper-parameters are tuned with Optuna [7] (TPE sampler) for 20 to 40 trials per setting, targeting validation F1-macro. Sample weights are balanced; three random seeds are averaged by soft-voting.

LSTM (bidirectional). Two stacked bidirectional LSTM [8] layers (hidden=128, dropout=0.3) followed by layer normalization and a linear head. We evaluate look-back windows of 20 and 60 trading days. The last hidden state feeds the classifier. Three seeds are averaged.

1-D CNN. Three Conv1D [9] blocks (filters 64-128-192, kernel=3, batch normalization (BN), ReLU activation, dropout) followed by global average + max pooling along the temporal axis. Two dense layers produce the logits. No recurrence is used.

CNN-LSTM. A CNN front-end (filters 64-128) extracts local patterns; its output is fed into an LSTM (hidden=128, two layers) that captures longer dependencies. A linear head produces the logits. The last five input channels carry locally normalized OHLCV (each window divided by its first close and max volume) so that the CNN learns scale-invariant candle shapes.

Training and selection. Inputs are scaled inside each split: `StandardScaler` for LR, `MinMaxScaler` for the deep models; XGBoost takes the raw values. The deep models train for up to 80 epochs with AdamW, gradient clipping at 1.0 and `ReduceLROnPlateau` on validation loss; early stopping with patience 15 is applied. The final per-run prediction is the average of three random-seed checkpoints (five for LR).

5 Evaluation

5.1 Classification

We use **F1-macro** as the primary metric: the unweighted mean of the per-class F1 scores. Macro averaging gives equal weight to the BUY, HOLD and SELL classes, which is appropriate given the deliberate near-balance of the label distribution. We report per-class F1 in the appendix.

5.2 Backtest

For every test day the predicted label is mapped to a position: BUY $\rightarrow$ +1, HOLD $\rightarrow$ 0, SELL $\rightarrow$ -1. Daily strategy return is $r_{\text{strat}}(t) = \text{position}(t) \cdot r_{\text{fwd}}(t)$. We report:

- **Win rate**: fraction of trading days with $r_{\text{strat}} > 0$ among the days that the model takes a position (HOLD days excluded).
- **Profit factor**: $\sum r^{+}_{\text{strat}} / |\sum r^{-}_{\text{strat}}|$, where the sums are over the same set of days.
- **Maximum drawdown**: minimum of $(\text{equity}_t - \text{peak}_t)/\text{peak}_t$ on the cumulative equity curve.
- **Annualized Sharpe ratio**: $\sqrt{252}\,\bar{r}_{\text{strat}}/\sigma_{r_{\text{strat}}}$, with $r_f = 0$.

We deliberately omit transaction costs and slippage; the assumption is a first-order comparison between models under identical conditions.

5.3 A note on accuracy

We do not report accuracy. With three near-equally-distributed classes, the classifier can reach ≈ 0.40 accuracy by predicting only HOLD, which is uninformative. F1-macro penalizes such collapse.

6 Statistical feature pre-selection: should we use it?

A common preliminary step is to filter features through statistical tests: Pearson correlation against the target, the Variance Inflation Factor (VIF) for multicollinearity (suited to LR), SHapley Additive exPlanations (SHAP) [10] rank (suited to tree models), or Spearman rank correlation (more appropriate for the non-linear models). We compared, on the same code path, every model trained with the full 61 features against the same model trained with only those features that passed its corresponding test on the validation set.

Table 3: Effect of statistical pre-selection on GLOBAL F1-macro: Δ = F1(all 61) − F1(filtered). A positive value means that using all features is better.

Model	Filter	Exp A	Exp B	Exp C
LR	Pearson + VIF (4–8 feats)	+0.024	+0.026	+0.021
XGBoost	SHAP top-N (23–25 feats)	+0.009	+0.021	+0.022
LSTM	Spearman (3–25 feats)	+0.039	+0.036	−0.006
CNN-1D	Spearman	+0.035	−0.025	+0.011
CNN-LSTM	Spearman	−0.011	+0.020	−0.026

Result. Table 3 shows that in 11 of 15 cases keeping all 61 features beats the filtered subset. The deltas are largest for LR (the model with the strongest internal regularization), where the elasticnet penalty performs implicit selection more effectively than the external Pearson/VIF screen. For XGBoost the SHAP-based filter is competitive (smallest deltas), as expected for a tree model. For the deep models the result depends on the experiment.

Decision. All subsequent results use the full 61 features for every model. The Pearson/VIF/Spearman/SHAP tests are still useful as exploratory tools (e.g. to communicate which indicators dominate), but they should not be used as a pre-training filter.

7 Results

We report 120 controlled runs (5 models × 3 splits × (1 global + 7 per-ticker)). Throughout this section "best lookback" is selected on the validation set when applicable (LSTM, CNN, CNN-LSTM).

7.1 Global models

Table 4 shows F1-macro on the 2025 test set when one single model is trained on the concatenated data of all seven tickers. LR leads on average (0.399) and wins two of the three experiments outright. XGBoost is the runner-up and edges LR by one point in Exp B. The deep models cluster lower, around 0.34–0.39.

The economic metrics tell a more nuanced story (Fig. 1 and Table 5). For the recommended experiment (Exp B), LR has the highest profit factor (1.21) and Sharpe (+0.76). XGBoost matches the F1 but its strategy is closer to random in economic terms. CNN-LSTM controls drawdown best (−0.40). The pure CNN is the weakest both in F1 and Sharpe.

7.2 Per-ticker models

When one model is trained per ticker (Table 6), LR is again the most consistent: it leads in all three experiments. The deep models drop more visibly compared to the global setting, because each per-ticker training set contains only ∼1500 samples. XGBoost remains a solid second.

Table 4: F1-macro on the 2025 test set, GLOBAL strategy. Best per column in bold.

Model	Exp A (10y)	Exp B (6y)	Exp C (4y)	Avg
LR	**0.411**	0.385	**0.401**	**0.399**
XGBoost	0.396	**0.386**	0.373	0.385
LSTM	0.389	0.370	0.370	0.376
CNN-LSTM	0.376	0.384	0.371	0.377
CNN 1D	0.361	0.343	0.373	0.359
Random 1/3	0.333	0.333	0.333	0.333

Table 5: Economic metrics on Exp B GLOBAL (test = 2025). Sharpe is annualized.

Model	F1	Win rate	Profit factor	Max drawdown	Sharpe
LR	0.385	0.497	**1.210**	−0.470	**+0.755**
XGBoost	**0.386**	0.500	1.083	−0.432	+0.345
CNN-LSTM	0.384	**0.505**	1.041	**−0.403**	+0.147
LSTM	0.370	0.491	1.120	−0.406	+0.433
CNN 1D	0.343	0.510	1.031	−0.434	+0.121

7.3 Where does the cross-asset variation come from?

Figure 2 breaks down the annualized Sharpe by model and ticker. Vertical bands are visible: AMZN and TSLA tend to be green across models, while GOOGL is red. This is a property of the assets, not of the models: the more directionally tradeable tickers (high realized vol, well-defined trends) reward any reasonable classifier, while range-bound tickers punish all of them. Averaging the per-model standard deviation across tickers (0.62 to 0.93) shows that the model contributes substantially less than the ticker to the total Sharpe variance.

7.4 Effect of the training-window length

The three splits share the same test year, so any difference between them is purely the effect of how many years of history the model can fit. Figure 4 suggests that LR benefits monotonically from longer histories (Exp A: 0.411 > Exp B: 0.385 > Exp C: 0.401, with a small Exp C rebound). The deep models are largely flat, indicating that more history does not help much when capacity is fixed and the optimizer is the bottleneck.

7.5 Signal distribution

Figure 5 shows the predicted distribution of BUY/HOLD/SELL labels for the five global models on Exp B. The base rate is roughly 30/40/30. CNN 1D col-

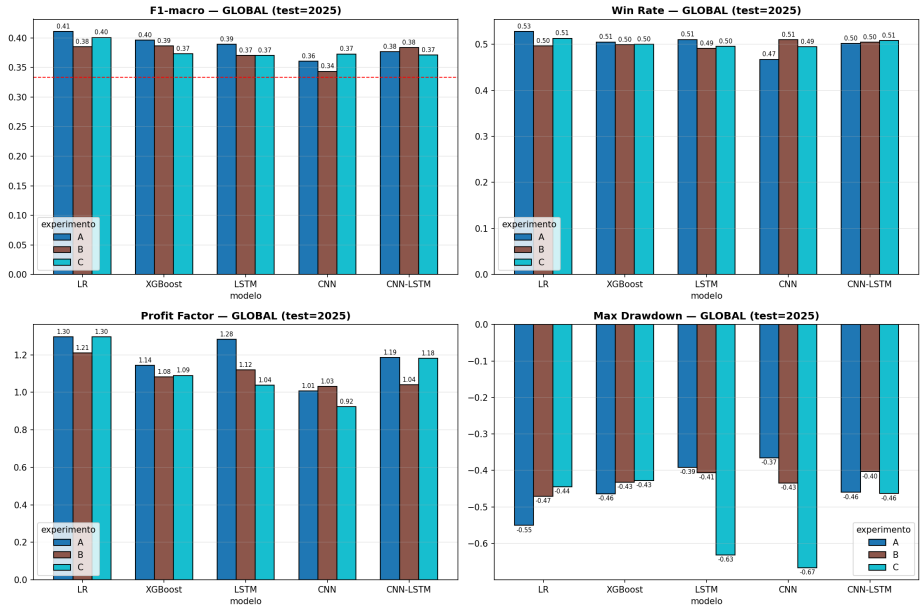

Fig. 1: Global models on the 2025 test set: F1-macro (top-left), win rate (top-right), profit factor (bottom-left) and maximum drawdown (bottom-right). The dashed red line in the F1 panel marks the $1/3$ random baseline. Higher is better except for drawdown.

lapses heavily towards BUY (49 %); LR is the most balanced but slightly conservative on BUY (18 %). This asymmetry is a useful diagnostic: a model whose predicted distribution is far from the prior is either over-confident on one side or has converged to a degenerate solution.

7.6 Drill-down on the best model per ticker

Table 7 reports the per-ticker breakdown of the highest F1 model in Exp B (XGBoost). TSLA is the outlier: F1 $= 0.44$, Sharpe $= +2.06$, $+201$ % cumulative return. The lowest performer is GOOGL with negative Sharpe.

8 Discussion

Why does LR win on F1? With approximately 1500 training samples per ticker (10 500 globally) and 61 inputs, deep models with 10^5–10^6 parameters are firmly in an overfitting regime. The technical indicators already capture most of the linear predictive content of price action; the elasticnet penalty selects an effective subset of them automatically and stabilizes the remaining coefficients. XGBoost shows similar resilience because tree pruning and the L2 leaf penalty play the same role.

Table 6: Average F1-macro across 7 tickers, PER-TICKER strategy. Best per column in bold.

Model	Exp A	Exp B	Exp C	Avg
LR	**0.411**	**0.392**	**0.359**	**0.387**
XGBoost	0.376	0.370	0.352	0.366
CNN-LSTM	0.368	0.342	0.356	0.355
LSTM	0.314	0.353	0.326	0.331
CNN 1D	0.378	0.332	0.280	0.330

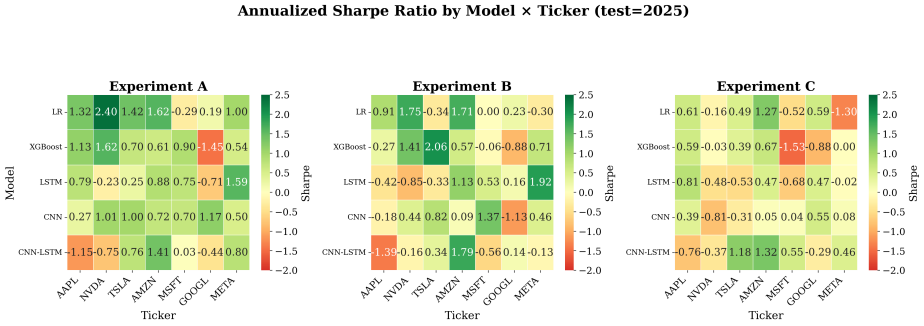

Fig. 2: Annualized Sharpe ratio per model and ticker, for the three experiments (test = 2025).

This matches a broader finding: on tabular data with engineered features, tree-based and linear models often match or beat deep networks, whose inductive biases target raw spatial/sequential signals, not engineered columns [13,14]; Gu et al. [15] report the same ordering in asset pricing at much larger scale.

Why is the cross-asset variation so large? Section 6.3 indicates that the model accounts for a smaller share of Sharpe variance than the ticker does. Tickers with persistent trends or high realized volatility (AMZN, TSLA, NVDA) reward any reasonably calibrated classifier; range-bound tickers (GOOGL) do not. This has practical consequences: a portfolio constructor would do better to pre-screen tickers by predictability than to choose a more elaborate model class.

Global vs. per ticker. Aggregating tickers into one global training set helps LR and XGBoost (Tables 4 and 6: 0.411 vs. 0.354 in Exp A, 0.385 vs. 0.392 in Exp B). The deep models do not benefit as much, presumably because the inter-ticker pattern they extract from the larger dataset is close to what each per-ticker model already finds.

When should one use the deep models? LSTM and CNN-LSTM produce competitive economic metrics on the volatile tickers, sometimes with smaller drawdown than LR. They become attractive when (i) the application requires modeling

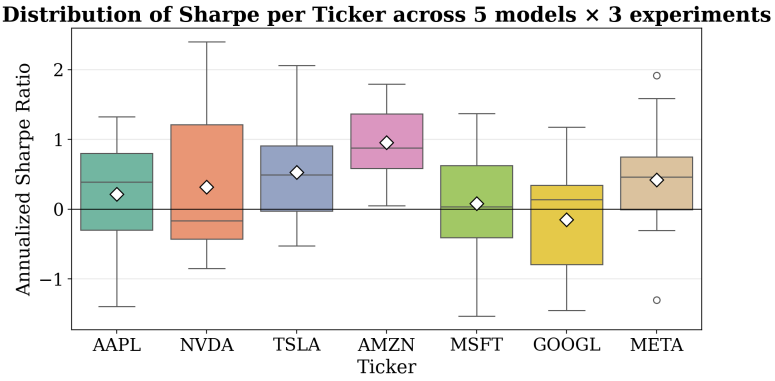

Fig. 3: Sharpe distribution per ticker over the 15 (model, experiment) combinations. Diamonds mark the mean. AMZN dominates; GOOGL is the only ticker with a negative median.

Table 7: Per-ticker drill-down for XGBoost Exp B GLOBAL (test = 2025).

Ticker	F1-macro	Sharpe	Cum. return	Win rate	Max drawdown
TSLA	**0.441**	**+2.06**	**+2.01**	0.604	−0.20
MSFT	0.399	−0.06	−0.01	0.473	−0.22
AMZN	0.390	+0.57	+0.19	0.531	−0.21
META	0.347	+0.71	+0.27	0.470	−0.18
NVDA	0.338	+1.41	+0.77	0.539	−0.37
AAPL	0.336	+0.27	+0.08	0.513	−0.24
GOOGL	0.336	−0.88	−0.21	0.487	−0.35

temporal dependencies beyond the look-back of the features themselves and (ii) enough training history is available to justify the parameter count. In our setting, neither condition is decisively met.

9 Limitations

We do not include transaction costs or slippage; under realistic costs the rank order between models with Sharpe < 0.5 would likely flip toward the strategy that trades least. The sample is restricted to seven large-cap US tech stocks, so conclusions need not generalize to other sectors or markets. Position sizing is binary (± 1 or 0); a Kelly or volatility-targeted scheme would change absolute Sharpe figures while keeping the relative ranking. We use a single split per experiment; walk-forward, purged cross-validation [12] would tighten confidence intervals at higher compute cost.

A consequence is that all three splits test on 2025, so a single period cannot separate a model effect from a year-specific regime; rolling the protocol

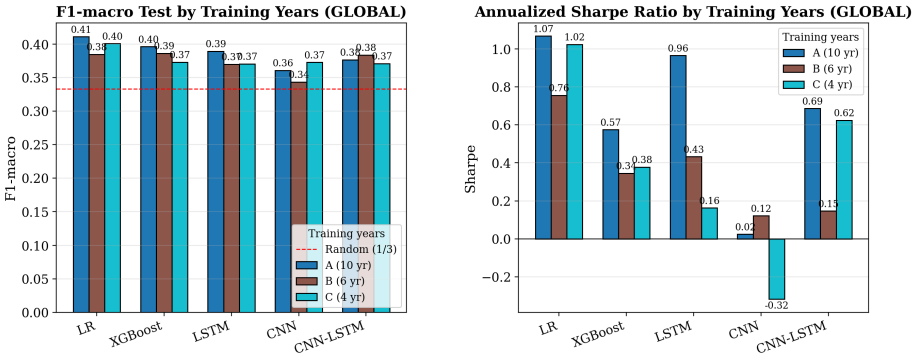

Fig. 4: Effect of training-window length on F1-macro (left) and Sharpe (right). All settings test on year 2025.

over several years (e.g. 2019–2025) is the most important extension for future work. Search is also asymmetric: XGBoost/LR get an automated/gridded search, deep models only hand-fixed sizes tuned over the look-back window — so the LR/XGBoost gap is a lower bound on what they could achieve with equal tuning.

10 Conclusions

A multinomial logistic regression with elasticnet regularization, trained on 61 indicator features derived from free Yahoo Finance data, beats three different deep architectures and a tuned XGBoost on average F1-macro across three temporal splits, and matches or exceeds them on the economic metrics. Statistical feature pre-selection (Pearson, VIF, Spearman, SHAP) does not help: keeping all features and relying on the model's own regularization wins in 11 out of 15 cases. Ticker volatility, not model class, explains the bulk of the cross-asset Sharpe variation. For practitioners working with daily large-cap data and no access to alternative datasets, a well-regularized linear classifier is a competitive and computationally cheap default; deep models become attractive only with longer histories or richer inputs.

Reproducibility. All code and data manifests are available in the supplementary material. Random seeds are fixed and listed per run; library versions match those in `requirements.txt`.

References

1. Fama, E. F.: Efficient capital markets: A review of theory and empirical work. The Journal of Finance **25**(2), 383–417 (1970)
2. Sezer, O. B., Gudelek, M. U., Ozbayoglu, A. M.: Financial time series forecasting with deep learning: A systematic literature review. Applied Soft Computing **90**, 106181 (2020)

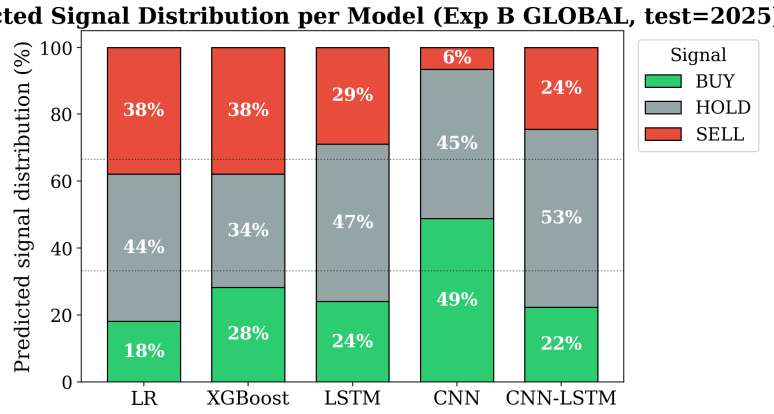

Fig. 5: Predicted distribution of the three signals for the global models on Exp B (test=2025). Dotted lines indicate the 33.3 % / 66.6 % references.

3. Fischer, T., Krauss, C.: Deep learning with long short-term memory networks for financial market predictions. European Journal of Operational Research **270**(2), 654–669 (2018)
4. Jiang, W.: Applications of deep learning in stock market prediction: recent progress. Expert Systems with Applications **184**, 115537 (2021)
5. Patel, J., Shah, S., Thakkar, P., Kotecha, K.: Predicting stock and stock price index movement using trend deterministic data preparation and machine learning techniques. Expert Systems with Applications **42**(1), 259–268 (2015)
6. Chen, T., Guestrin, C.: XGBoost: A scalable tree boosting system. In: Proc. KDD, pp. 785–794. ACM (2016)
7. Akiba, T., Sano, S., Yanase, T., Ohta, T., Koyama, M.: Optuna: A next-generation hyperparameter optimization framework. In: Proc. KDD, pp. 2623–2631 (2019)
8. Hochreiter, S., Schmidhuber, J.: Long short-term memory. Neural Computation **9**(8), 1735–1780 (1997)
9. LeCun, Y., Boser, B., Denker, J. S., et al.: Backpropagation applied to handwritten zip code recognition. Neural Computation **1**(4), 541–551 (1989)
10. Lundberg, S. M., Lee, S.-I.: A unified approach to interpreting model predictions. In: Advances in Neural Information Processing Systems, pp. 4765–4774 (2017)
11. Zou, H., Hastie, T.: Regularization and variable selection via the elastic net. Journal of the Royal Statistical Society B **67**(2), 301–320 (2005)
12. López de Prado, M.: Advances in Financial Machine Learning. Wiley (2018)
13. Grinsztajn, L., Oyallon, E., Varoquaux, G.: Why do tree-based models still outperform deep learning on tabular data? In: Advances in Neural Information Processing Systems (Datasets and Benchmarks Track), vol. 35 (2022)
14. Shwartz-Ziv, R., Armon, A.: Tabular data: Deep learning is not all you need. Information Fusion **81**, 84–90 (2022)
15. Gu, S., Kelly, B., Xiu, D.: Empirical asset pricing via machine learning. The Review of Financial Studies **33**(5), 2223–2273 (2020)